

Escopo de variáveis e regiões críticas com OpenMP

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Implementar em C uma estimativa de π pelo método de Monte Carlo. O programa deve sortear pontos em um quadrado de lado 2 e contar os que pertencem ao círculo unitário. A atividade pede a paralelização do laço principal com OpenMP, a identificação de uma condição de corrida no contador e duas formas de correção: uma seção `critical` a cada incremento e um acumulador local por thread. Também são exploradas as cláusulas `shared`, `private`, `firstprivate`, `lastprivate` e `default(none)`.

2 Fundamentação teórica

2.1 Método de Monte Carlo

O quadrado de lado 2 tem área 4 e o círculo unitário inscrito tem área π . Se os pontos forem uniformes no quadrado, a fração de pontos dentro do círculo converge para a razão entre as áreas:

$$\frac{n_{\text{dentro}}}{N} \approx \frac{\pi}{4}.$$

A estimativa calculada pelo programa é, portanto,

$$\hat{\pi} = 4 \cdot \frac{n_{\text{dentro}}}{N}.$$

Cada ponto (x, y) com $x, y \in [-1, 1]$ pertence ao círculo quando $x^2 + y^2 \leq 1$. Como o método usa amostras aleatórias, o resultado é uma aproximação. Aumentar N reduz a variação, mas não elimina o caráter estocástico da estimativa.

2.2 Condição de corrida

A operação `dentro_circulo++` não é atômica no nível da máquina. O processador executa três passos separados: ler o valor da memória para um registrador (LOAD), incrementar o registrador (ADD) e escrever o resultado de volta (STORE). Se duas threads executam o LOAD antes que qualquer uma complete o STORE, ambas leem o mesmo valor, ambas incrementam para o mesmo resultado e uma das escritas sobrescreve a outra. Perde-se um incremento. O programa termina normalmente, mas o resultado final é menor que o correto.

A gravidade do problema depende do número de threads e da frequência dos acessos concorrentes. Não há como prever exatamente quantos incrementos serão perdidos em cada execução, pois o escalonamento das threads e a ordem das operações mudam a cada execução.

2.3 Escopo de variáveis em OpenMP

Em uma região paralela, o OpenMP define regras de visibilidade para cada variável:

- `shared(var)`: todas as threads compartilham o mesmo endereço de memória. Alterações são visíveis a todas as threads, mas sem sincronização podem causar condições de corrida.
- `private(var)`: cada thread recebe uma cópia local da variável, sem valor inicial definido (conteúdo indeterminado). A variável externa não é modificada pela região paralela.
- `firstprivate(var)`: como `private`, mas a cópia local é inicializada com o valor que a variável tinha antes da região paralela.
- `lastprivate(var)`: aplicável a laços. Ao final do laço, a variável externa recebe o valor produzido na última iteração lógica, e não o valor da thread que terminou por último.
- `default(none)`: proíbe escopos implícitos. O compilador exige que toda variável referenciada dentro da região paralela seja explicitamente declarada com alguma cláusula de escopo. Variáveis esquecidas geram erro de compilação.

3 Implementação

A versão sequencial, `e0.c`, usa uma semente única e atualizada por `rand_r`. Ela fornece a referência para o cálculo. Em `e1.c`, cada índice do laço inicializa uma semente determinística e independente. Assim, as execuções recebem os mesmos pontos e a variação observada vem da condição de corrida no contador compartilhado.

Em `e2.c`, a diretiva `critical` protege somente o incremento de `dentro_circulo`. O sorteio e o teste geométrico ainda ocorrem em paralelo, mas todas as threads disputam a mesma seção crítica milhões de vezes.

A versão `e3.c` separa as diretivas `parallel` e `for`. Cada thread declara `local_dentro` e uma semente própria antes do laço. Durante o `for`, ela atualiza apenas o contador local. Depois do laço, cada thread entra uma vez em `critical` para somar seu resultado no contador global. A diretiva também usa `default(none)` e declara `total_elementos` e `dentro_circulo` como `shared`.

Para validar isoladamente as cláusulas de compartilhamento e escopo de dados em OpenMP, desenvolvemos quatro programas complementares:

- **`e4_private.c`** – demonstra a criação de cópias locais sem inicialização;
- **`e4_firstprivate.c`** – demonstra a inicialização da cópia local com o valor prévio;
- **`e4_lastprivate.c`** – demonstra a propagação do valor da última iteração lógica;
- **`e4_default_none.c`** – demonstra a obrigatoriedade de classificação explícita.

```
1 #define _DEFAULT_SOURCE
2
3 #include <stdio.h>
```

```

4 #include <stdlib.h>
5 #include <time.h>
6 #include <math.h>
7
8 #define TOTAL_ELEMENTOS_PADRAO 5000000L
9
10 int main(int argc, char *argv[]) {
11     long long total_elementos = argc > 1 ? atoll(argv[1]) :
        TOTAL_ELEMENTOS_PADRAO;
12
13     long long dentro_circulo = 0;
14
15     unsigned int seed = time(NULL);
16
17     for (long long i = 0; i < total_elementos; i++) {
18         double x = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
19         double y = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
20
21         if ((x * x + y * y) <= 1.0) {
22             dentro_circulo++;
23         }
24     }
25
26     double pi = 4.0 * dentro_circulo / total_elementos;
27     double erro = fabs(pi - M_PI);
28
29     printf("Dentro do circulo = %lld\nPI = %.10f\nErro = %.10e\n",
        dentro_circulo, pi, erro);
30
31     return 0;
32 }

```

Listing 1: e0.c, versão sequencial

```

1 #define _DEFAULT_SOURCE
2
3 /*
4  * dentro_circulo++ envolve ler, somar 1 e escrever de volta.
5  * Sao pelo menos 3 instrucoes de maquina que podem se entrelacar
6  * entre threads, perdendo incrementos.
7  */
8
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <math.h>
12 #include <omp.h>
13
14 #define TOTAL_ELEMENTOS_PADRAO 5000000L
15
16 int main(int argc, char *argv[]) {

```

```

17     long long total_elementos = argc > 1 ? atoll(argv[1]) :
        TOTAL_ELEMENTOS_PADRAO;
18
19     long long dentro_circulo = 0;
20
21     #pragma omp parallel for
22     for (long long i = 0; i < total_elementos; i++) {
23         unsigned int seed = (unsigned int)(i + 1);
24         double x = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
25         double y = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
26
27         if ((x * x + y * y) <= 1.0) {
28             dentro_circulo++;
29         }
30     }
31
32     double pi = 4.0 * dentro_circulo / total_elementos;
33     double erro = fabs(pi - M_PI);
34
35     printf("Dentro do circulo = %lld\nPI = %.10f\nErro = %.10e\n",
        dentro_circulo, pi, erro);
36
37     return 0;
38 }

```

Listing 2: e1.c, versão com condição de corrida

```

1  #define _GNU_SOURCE
2  #define _POSIX_C_SOURCE 199309L
3
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <math.h>
7  #include <omp.h>
8  #include <time.h>
9
10 #define TOTAL_ELEMENTOS_PADRAO 5000000L
11
12 static double obter_tempo_segundos(void) {
13     struct timespec tempo_atual;
14     clock_gettime(CLOCK_MONOTONIC, &tempo_atual);
15     return (double)tempo_atual.tv_sec + (double)tempo_atual.tv_nsec / 1e9;
16 }
17
18 int main(int argc, char *argv[]) {
19     long long total_elementos = argc > 1 ? atoll(argv[1]) :
        TOTAL_ELEMENTOS_PADRAO;
20
21     long long dentro_circulo = 0;
22

```

```

23     double tempo = obter_tempo_segundos();
24
25     #pragma omp parallel for
26     for (long long i = 0; i < total_elementos; i++) {
27         unsigned int seed = (unsigned int)(i + 1);
28         double x = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
29         double y = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
30
31         if ((x * x + y * y) <= 1.0) {
32             #pragma omp critical
33             {
34                 dentro_circulo++;
35             }
36         }
37     }
38
39     double pi = 4.0 * dentro_circulo / total_elementos;
40     double erro = fabs(pi - M_PI);
41
42     tempo = obter_tempo_segundos() - tempo;
43
44     printf("PI = %.10f\nErro = %.10e\nTempo = %.9f segundos\n", pi, erro,
45           tempo);
46
47     return 0;

```

Listing 3: e2.c, incremento protegido por critical

```

1  #define _GNU_SOURCE
2  #define _POSIX_C_SOURCE 199309L
3
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <math.h>
7  #include <omp.h>
8  #include <time.h>
9
10 #define TOTAL_ELEMENTOS_PADRAO 5000000L
11
12 static double obter_tempo_segundos(void) {
13     struct timespec tempo_atual;
14     clock_gettime(CLOCK_MONOTONIC, &tempo_atual);
15     return (double)tempo_atual.tv_sec + (double)tempo_atual.tv_nsec / 1e9;
16 }
17
18 int main(int argc, char *argv[]) {
19     long long total_elementos = argc > 1 ? atoll(argv[1]) :
20         TOTAL_ELEMENTOS_PADRAO;

```

```

21     long long dentro_circulo = 0;
22
23     double tempo = obter_tempo_segundos();
24
25     #pragma omp parallel default(none) shared(total_elementos,
26         dentro_circulo)
27     {
28         long long local_dentro = 0;
29         unsigned int seed = 123456789U ^ (unsigned int)omp_get_thread_num
30         ();
31
32         #pragma omp for
33         for (long long i = 0; i < total_elementos; i++) {
34             double x = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX -
35             1.0;
36             double y = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX -
37             1.0;
38
39             if ((x * x + y * y) <= 1.0) {
40                 local_dentro++;
41             }
42
43             #pragma omp critical
44             {
45                 dentro_circulo += local_dentro;
46             }
47
48             #pragma omp critical
49             {
50                 dentro_circulo += local_dentro;
51             }
52
53             double pi = 4.0 * dentro_circulo / total_elementos;
54             double erro = fabs(pi - M_PI);
55             tempo = obter_tempo_segundos() - tempo;
56
57             printf("PI = %.10f\nErro = %.10e\nTempo = %.9f segundos\n", pi, erro,
58                 tempo);
59
60             return 0;
61     }

```

Listing 4: e3.c, contador local por thread

```

1  #include <omp.h>
2  #include <stdio.h>
3
4  int main() {
5      int x = 100;
6
7      #pragma omp parallel for private(x)
8      for (int i = 0; i < 8; i++) {
9          printf("Thread %d: x (antes) = %d\n", omp_get_thread_num(), x);

```

```

10     x = omp_get_thread_num();
11     printf("Thread %d: x (depois) = %d\n", omp_get_thread_num(), x);
12 }
13
14 printf("Fora da regioao paralela: x = %d\n", x);
15 return 0;
16 }

```

Listing 5: e4_private.c, demonstração da cláusula private

```

1 #include <omp.h>
2 #include <stdio.h>
3
4 int main() {
5     int x = 100;
6
7     #pragma omp parallel for firstprivate(x)
8     for (int i = 0; i < 8; i++) {
9         printf("Thread %d: x (antes) = %d\n", omp_get_thread_num(), x);
10        x = omp_get_thread_num();
11        printf("Thread %d: x (depois) = %d\n", omp_get_thread_num(), x);
12    }
13
14    printf("Fora da regioao paralela: x = %d\n", x);
15    return 0;
16 }

```

Listing 6: e4_firstprivate.c, demonstração da cláusula firstprivate

```

1 #include <omp.h>
2 #include <stdio.h>
3
4 int main() {
5     int resultado = 0;
6
7     #pragma omp parallel for lastprivate(resultado)
8     for (int i = 0; i < 20; i++) {
9         resultado = i * i;
10        printf("Thread %d: i = %d, resultado = %d\n", omp_get_thread_num(),
11              , i, resultado);
12    }
13
14    printf("Apos o laco: resultado = %d (esperado: %d)\n", resultado, 19 *
15          19);
16    return 0;
17 }

```

Listing 7: e4_lastprivate.c, demonstração da cláusula lastprivate

```

1 #include <omp.h>

```

```

2  #include <stdio.h>
3
4  int main() {
5      int x = 10;
6      int y = 20;
7      int soma = 0;
8
9      #pragma omp parallel for default(none) shared(x, soma)
10     for (int i = 0; i < 10; i++) {
11         soma += x + y;  // 'y' nao esta classificada!
12     }
13
14     printf("soma = %d\n", soma);
15     return 0;
16 }

```

Listing 8: e4_default_none.c, demonstração de default(none)

Compilamos os códigos-fonte com GCC 13.3, padrão C11, nível de otimização -O2, avisos ativados (-Wall -Wextra) e suporte a OpenMP (-fopenmp). Para as medições de desempenho, a afinidade das threads é controlada por variáveis de ambiente (OMP_PROC_BIND=close e OMP_PLACES=cores), variando a contagem de threads de 1 até o limite da máquina com dez repetições por configuração.

As flags de compilação têm as seguintes funções: -std=c11 fixa o padrão C11 da linguagem; -O2 aplica otimizações do compilador; -Wall -Wextra ativam diagnósticos adicionais; -fopenmp habilita a interpretação de diretivas OpenMP e a vinculação com a libgomp; e -lm vincula a biblioteca matemática padrão.

```

1  gcc -std=c11 -O2 -Wall -Wextra -fopenmp e0.c -o e0 -lm
2  gcc -std=c11 -O2 -Wall -Wextra -fopenmp e1.c -o e1 -lm
3  gcc -std=c11 -O2 -Wall -Wextra -fopenmp e2.c -o e2 -lm
4  gcc -std=c11 -O2 -Wall -Wextra -fopenmp e3.c -o e3 -lm
5
6  OMP_NUM_THREADS=28 ./e1 5000000
7  OMP_NUM_THREADS=28 ./e2 5000000
8  OMP_NUM_THREADS=28 ./e3 5000000

```

Listing 9: Exemplo de compilação e execução

4 Resultados e discussão

Realizamos os testes com $N = 5.000.000$, dez repetições por configuração, em um Intel Core i7-14700HX exposto ao WSL2 com 14 núcleos físicos e 28 threads lógicas. A afinidade foi configurada com `OMP_PROC_BIND=close` e `OMP_PLACES=cores`, e `OMP_DYNAMIC` ficou desativado. A máquina é um laptop sob um hipervisor – o sistema operacional interfere nas medições e o desvio padrão é alto em várias configurações. As tabelas usam a mediana dos tempos; as barras de erro dos gráficos mostram o desvio padrão das dez repetições.

4.1 Versão sem sincronização

Com uma thread, e1.c retornou $\hat{\pi} = 3,1415096$, igual ao resultado de e2.c. Os dois programas usam a mesma sequência de pontos, portanto a diferença a partir de duas threads só pode vir do acesso concorrente ao contador. A Tabela 1 resume as dez execuções de e1.c. Com 2 threads, a média já cai para 2,06 (erro de 34%). Com 4 threads, cai para 0,91 (erro de 71%). Com 8 threads ou mais, a média oscila entre 0,38 e 0,62, sempre muito abaixo do valor correto.

Table 1: Estimativa de π em e1.c com race condition (10 execuções por configuração).

Threads	Média de $\hat{\pi}$	Desvio padrão	Intervalo observado
1	3,1415096	0,0000000	[3,1415096; 3,1415096]
2	2,0640822	0,1508933	[1,8089600; 2,2215192]
4	0,9132916	0,0891818	[0,7157944; 1,0165576]
8	0,4566514	0,0802879	[0,3161328; 0,5506096]
14	0,4859702	0,1299359	[0,3248536; 0,6791216]
16	0,4114073	0,1164830	[0,2833832; 0,6941400]
28	0,4478194	0,0733959	[0,3476832; 0,5515736]

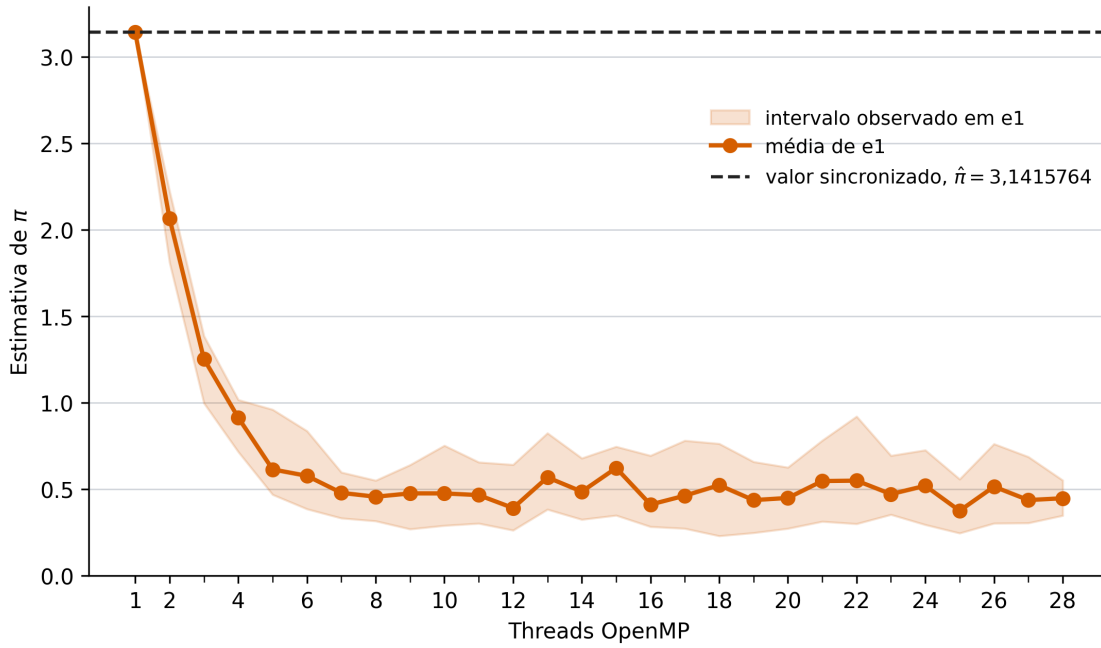


Figure 1: Média e intervalo das dez execuções de e1.c. A linha tracejada indica a estimativa sincronizada com os mesmos pontos.

A Figura 1 mostra que o efeito não segue uma curva regular. A ordem das leituras e escritas muda em cada execução, portanto não há como prever quantos incrementos serão perdidos. O programa compila e termina sem falha. A comparação com uma versão sincronizada é o que revela o defeito.

4.2 Custo da seção crítica

A Tabela 2 compara os tempos de e2.c e e3.c. Em e2.c, o tempo mediano subiu de 0,096032 s com uma thread para 1,221722 s com 28. O trabalho geométrico é paralelo, mas aproximadamente $\pi N/4$, ou 3,93 milhões de pontos, entram no círculo e disputam a seção crítica. Com mais threads, a disputa cresce e o tempo flutua entre 0,9 e 1,5 s desde a oitava thread, com desvios padrão altos. O speedup fica abaixo de 1 em todas as configurações com mais de uma thread: a paralelização *piora* o desempenho.

Table 2: Tempo mediano em segundos. A razão compara as medianas de e2.c e e3.c.

Threads	e2, critical por ponto	e3, contador local	Razão e2/e3
1	0,096032	0,055613	1,73×
2	0,318125	0,027523	11,56×
4	0,570798	0,014342	39,80×
8	0,902719	0,009420	95,83×
14	0,746605	0,003498	213,44×
16	0,902004	0,003321	271,58×
28	1,221722	0,009430	129,56×

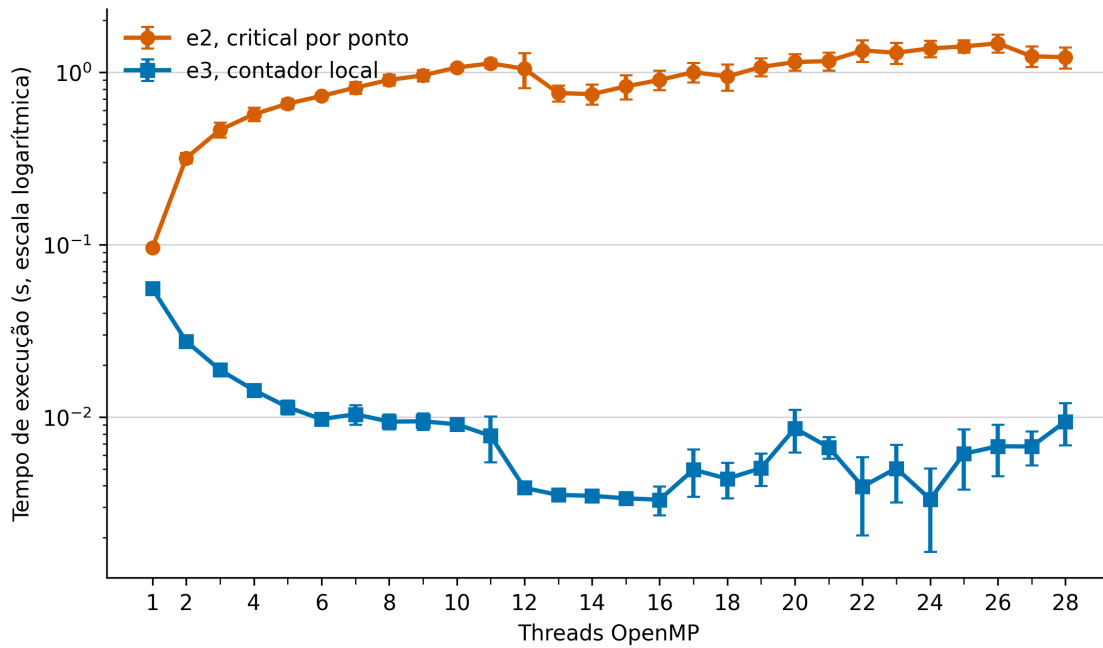


Figure 2: Tempo mediano de execução. As barras de erro mostram o desvio padrão de dez repetições. A escala vertical é logarítmica.

Em e3.c, cada thread entra em `critical` uma vez, depois de processar o seu bloco de iterações. O tempo caiu de 0,055613 s com uma thread para 0,003498 s com 14 threads, um ganho de 15,9×. Com 16 threads, o tempo foi 0,003321 s, o menor valor medido. Nas duas configurações o ganho é próximo de 16, a quantidade de threads lógicas. Com 28 threads, o tempo voltou para 0,009430 s. Nesta máquina, o melhor resultado ocorreu com os 14 núcleos físicos ou logo acima deles.

4.3 Comportamento das cláusulas de escopo de dados

Executamos quatro programas complementares para avaliar o comportamento isolado de cada cláusula de escopo:

- `private(x)` (e4_private.c): cada thread recebe uma instância não inicializada de `x` em sua pilha privada. A leitura de `x` antes de qualquer atribuição resulta em valor indefinido (lixo de memória). Ao final da região paralela, a variável externa preserva o valor original (100).
- `firstprivate(x)` (e4_firstprivate.c): cada thread recebe uma cópia local inicializada com o valor prévio da variável (100). Modificações internas afetam apenas o escopo local de cada thread, e o valor fora da região paralela permanece 100.
- `lastprivate(resultado)` (e4_lastprivate.c): ao término do laço paralelo de 20 iterações ($i \in [0, 19]$), a variável externa recebe deterministicamente o valor computado na última iteração lógica ($19^2 = 361$), independentemente da thread que finalizou por último.
- `default(none)` (e4_default_none.c): ao omitir a classificação da variável `y`, o compilador GCC detecta a variável não declarada e aborta a compilação:

```
1 error: 'y' not specified in enclosing 'parallel'
```

Esse mecanismo estático previne efeitos colaterais e corridas acidentais em programas paralelos complexos.

5 Conclusão

A versão e1.c mostra que um contador compartilhado não pode receber incrementos concorrentes sem sincronização: o programa compila, executa e imprime um valor, mas a estimativa de π com 8 threads cai para 0,46 contra o valor correto de 3,14. e2.c preserva a contagem, mas serializa milhões de atualizações e perde desempenho conforme o número de threads aumenta. e3.c acumula localmente e sincroniza apenas no final. Com 14 threads, a versão levou 0,003498 s, contra 0,746605 s de e2.c na mesma configuração – o speedup atinge $16\times$, contra $0,13\times$ de e2.c. O teste também mostra o limite prático do hardware: depois dos 14 núcleos físicos, as threads lógicas não melhoraram o tempo de e3.c e ainda aumentaram a variância das medições.

As cláusulas `private`, `firstprivate`, `lastprivate` e `default(none)` controlam o escopo e a visibilidade das variáveis. A cláusula `default(none)` se torna útil em programas complexos porque obriga a declaração explícita de todas as variáveis, prevenindo erros sutis de compartilhamento acidental.